

Trabalho – Engenharia de Software I

Guilherme Nogueira Zanon

João Lucas Oliveira

Desenvolvimento de um Framework C++ para Construção de Simulações Baseadas na Dinâmica de Sistemas

Questão 1 – Casos de Uso e Critérios de Aceitação

A API desenvolvida possui como principal objetivo permitir a construção e execução de modelos baseados na Teoria Geral de Sistemas, conforme apresentado no enunciado do trabalho. A estrutura implementada foi baseada nos conceitos de sistemas, fluxos e modelos dinâmicos, permitindo representar diferentes cenários por meio de orientação a objetos em C++.

O primeiro caso de uso identificado consiste na criação de sistemas. Os sistemas representam estoques ou acumuladores capazes de armazenar uma determinada quantidade numérica ao longo da simulação. Esses sistemas podem representar populações, quantidade de água, energia, recursos financeiros ou qualquer outra variável acumulativa. O usuário deve ser capaz de criar sistemas informando um nome e um valor inicial. O principal critério de aceitação desse caso de uso é garantir que os valores armazenados sejam corretamente mantidos e acessados pelos métodos da classe.

Outro caso de uso importante é a criação de fluxos. Os fluxos representam mecanismos de transferência ou alteração dos sistemas durante a simulação. Cada fluxo possui um sistema de origem e um sistema de destino, sendo responsável por calcular uma quantidade que será posteriormente aplicada pelo modelo. O critério de aceitação associado a esse caso de uso consiste em garantir que o fluxo seja capaz de acessar corretamente seus sistemas associados e produzir resultados coerentes de acordo com sua equação.

Também foi identificado o caso de uso referente à execução do modelo exponencial. Nesse modelo, o crescimento populacional ocorre de maneira proporcional à população atual, seguindo a equação de crescimento exponencial apresentada no enunciado. O critério de aceitação consiste em verificar se a população cresce exponencialmente ao longo do tempo e se os resultados obtidos são compatíveis com os valores apresentados no software Vensim.

Outro caso de uso implementado foi o modelo logístico. Nesse modelo, além da taxa de crescimento, também existe uma limitação ambiental representada pela variável Pmax,

correspondente à capacidade máxima de suporte do ambiente. O comportamento esperado é um crescimento inicial acelerado seguido de estabilização próxima ao valor de P_{max} . O principal critério de aceitação desse modelo é verificar se a população realmente converge para a capacidade máxima estabelecida.

Por fim, foi identificado o caso de uso referente à execução temporal da simulação. O modelo deve ser capaz de executar um loop temporal no qual todos os fluxos são calculados e posteriormente aplicados aos sistemas. O critério de aceitação desse processo consiste em garantir que os fluxos sejam executados corretamente a cada iteração e que os sistemas sejam atualizados conforme o algoritmo geral apresentado no enunciado.

Questão 2 – Validação dos Casos de Uso e Critérios de Aceitação

A validação dos casos de uso e critérios de aceitação foi realizada com base nos modelos fornecidos no software Vensim. O arquivo de validação disponibilizado no trabalho foi utilizado como referência para análise do comportamento esperado dos modelos exponencial e logístico.

Os resultados produzidos pela API desenvolvida foram comparados com os resultados observados no Vensim, permitindo verificar a coerência matemática e comportamental das simulações. Durante os testes realizados, o modelo exponencial apresentou crescimento contínuo e acelerado, enquanto o modelo logístico apresentou crescimento limitado pela capacidade máxima do ambiente, estabilizando-se próximo ao valor de P_{max} .

Além disso, a validação também foi realizada por meio de testes funcionais automatizados utilizando a biblioteca padrão do C++. Esses testes permitiram verificar o correto funcionamento das classes System, Flow, ExponentialFlow, LogisticFlow e Model, garantindo que os critérios de aceitação definidos anteriormente fossem satisfeitos.

Questão 3 – Cenários de Teste

Diversos cenários de teste foram definidos durante o desenvolvimento da API com o objetivo de validar o comportamento esperado do framework. Esses cenários foram implementados na forma de testes automatizados utilizando a função `assert()`.

O primeiro cenário de teste consistiu na validação da classe System. Nesse teste, um sistema foi criado com valor inicial igual a 100. Em seguida, verificou-se se o método `getValue()` retornava corretamente o valor armazenado. Posteriormente, o valor foi alterado para 200 utilizando o método `setValue()`, sendo realizada uma nova verificação.

Outro cenário importante foi a validação do fluxo exponencial. Nesse teste, foi criada uma população inicial igual a 100 e executado um fluxo exponencial responsável por calcular

30% do valor atual da população. O resultado esperado para o fluxo era igual a 30.

Também foi implementado um cenário de teste para o modelo exponencial completo. Nesse caso, o modelo foi executado durante dez iterações temporais partindo de uma população inicial igual a 10. O resultado final esperado foi compatível com o crescimento exponencial previsto matematicamente.

No caso do modelo logístico, foram implementados testes para verificar o cálculo correto da equação logística e também o comportamento de estabilização da população. Utilizando uma população inicial igual a 10 e um Pmax igual a 70, observou-se que a população crescia inicialmente e posteriormente convergia para valores próximos à capacidade máxima do ambiente.

Esses cenários de teste permitiram validar tanto o funcionamento individual das classes quanto a integração completa do framework.

Questão 4 – Projeto UML da API

A modelagem da API foi realizada utilizando os princípios da orientação a objetos e representada por meio de diagramas UML. As principais classes identificadas durante o projeto foram System, Flow, ExponentialFlow, LogisticFlow e Model.

A classe System foi definida como responsável pelo armazenamento das quantidades numéricas utilizadas na simulação. Essa classe possui atributos relacionados ao nome do sistema e ao valor armazenado.

A classe Flow foi projetada como uma classe abstrata responsável por representar fluxos genéricos entre sistemas. Ela possui referências para os sistemas de origem e destino e define a função virtual pura execute(), responsável por permitir polimorfismo entre diferentes tipos de fluxo.

As classes ExponentialFlow e LogisticFlow herdam da classe Flow e implementam diferentes equações matemáticas de crescimento populacional. Essa estrutura permitiu reutilização de código e flexibilidade na criação de novos modelos.

A classe Model foi desenvolvida para representar o simulador principal. Ela armazena listas de sistemas e fluxos e executa o loop temporal responsável pela simulação.

O relacionamento entre as classes foi definido da seguinte forma: as classes ExponentialFlow e LogisticFlow possuem herança em relação à classe Flow; a classe Flow possui associação com System; e a classe Model possui agregação de múltiplos sistemas e fluxos.

